

实验四：校园共享单车检测

一、实验目的

1. 掌握目标检测的基本概念和工作流程
2. 理解 YOLO (You Only Look Once) 系列算法的原理
3. 学习使用预训练模型进行迁移学习
4. 了解 COCO 数据集的结构和使用方法
5. 掌握目标检测结果的可视化和评估方法

二、实验原理

2.1 目标检测概述

目标检测是计算机视觉的核心任务之一，需要同时完成：

1. 目标定位 (Localization)：确定目标在图像中的位置 (边界框)
2. 目标分类 (Classification)：识别目标的类别

2.2 YOLO 算法原理

YOLO (You Only Look Once) 是一种单阶段目标检测算法，其核心思想是将目标检测视为回归问题：

1. 网格划分：将输入图像划分为 $S \times S$ 的网格
2. 边界框预测：每个网格预测 B 个边界框及其置信度
3. 类别预测：每个网格预测 C 个类别的条件概率
4. 非极大值抑制 (NMS)：去除重叠的检测框

2.3 YOLOv8 模型结构详细解析

本实验使用 YOLOv8n 作为基础模型。YOLOv8 整体可划分为 Backbone、Neck、Head 三个部分，输入分辨率以 640×640 为例：

2.3.1 Backbone：特征提取网络

Backbone 负责从原始图像中提取多尺度语义特征，YOLOv8 主要采用 C2f 模块 + SPPF 结构：

1. Stem 卷积层：对输入图像进行初步卷积与下采样，得到较低分辨率但更高通道数的特征图；
2. C2f 模块：
 - 由多分支卷积和残差连接组成，类似轻量化的 CSP/ResNet 结构；
 - 通过特征分裂 (split) 与融合 (fusion) 实现更丰富的梯度流；
3. SPPF (Spatial Pyramid Pooling-Fast)：
 - 使用多尺度最大池化 (如 5×5 、 9×9 、 13×13) 抽取不同感受野的信息；
 - 将结果拼接后再卷积，获得对目标尺度更鲁棒的高级特征。

不同尺度的特征图形状示例：

层级	典型输出尺寸 (C×H×W)	含义
P3	128×80×80	细粒度、小目标特征
P4	256×40×40	中等尺度目标
P5	512×20×20	大目标与全局语义

2.3.2 Neck: 特征融合 (FPN + PAN)

Neck 负责在不同尺度之间进行自顶向下 (FPN) 与自底向上 (PAN) 特征融合:

1. FPN (Feature Pyramid Network):
 - 将高层语义特征上采样, 与低层细节特征相加 / 拼接;
 - 使得小目标也能“看到”更强的语义信息;
2. PAN (Path Aggregation Network):
 - 再次自底向上融合, 强化定位信息;
 - 保持高分辨率特征中的边缘与纹理细节。

通过 Neck, YOLOv8 获得了三层多尺度特征 (通常对应 80×80 、 40×40 、 20×20 的特征图), 分别用于检测不同尺度的目标。

2.3.3 Head: 解耦检测头 (Anchor-Free)

YOLOv8 的 Detection Head 采用 解耦结构 + Anchor-Free 设计:

1. 解耦头 (Decoupled Head):
 - 对每个尺度的特征图, 分成两条分支:
 - 分类分支: 预测每个位置属于某一类别的概率;
 - 回归分支: 预测边界框的中心、宽高等几何信息;
 - 相比耦合头, 更有利于优化两个任务, 提升精度。
2. Anchor-Free 预测:
 - 不再依赖一组预定义的锚框 (anchor), 而是直接预测相对网格点的中心偏移和宽高;
 - 降低超参数数量, 减少对先验框设计的依赖, 对小目标和特殊纵横比目标更友好。

总体来看, YOLOv8 的结构在保持 YOLO 系列高效实时特性的同时, 通过 C2f、SPPF、解耦头和 Anchor-Free 设计, 在 精度与速度之间取得了更好的平衡。

2.4 COCO 数据集

COCO (Common Objects in Context) 是目标检测领域最常用的数据集之一:

- 包含 80 个目标类别
- 超过 20 万张标注图像
- 自行车 (bicycle) 的类别 ID 为 1

2.5 迁移学习

使用在 COCO 数据集上预训练的模型权重, 可以直接用于检测自行车等 COCO 包含的类别, 无需额外训练。

2.6 检测流程图 (从输入到输出)

输入图像 → 预处理 → YOLOv8 网络前向推理 → 候选框解码 → 置信度筛选 → NMS 去重 → 过滤出自行车 → 可视化与结果统计

三、实验方法

3.1 检测器设计

```
class BicycleDetector:
    """共享单车检测器"""
    def __init__(self, model_path='yolov8n.pt'):
        print(f"正在加载模型: {model_path}")
        self.model = YOLO(model_path)
        # COCO数据集中自行车的类别ID是1 (bicycle)
        self.bicycle_class_id = 1

    def detect_bicycles(self, image_path, conf_threshold=0.3):
        print(f"\n正在检测图像: {image_path}")
        # 使用YOLO模型进行检测
        results = self.model(image_path, conf=conf_threshold)
        # 提取自行车检测结果
        detections = []
        for result in results:
            boxes = result.boxes
            for box in boxes:
                # 获取类别ID
                class_id = int(box.cls[0])

                # 只保留自行车类别
                if class_id == self.bicycle_class_id:
                    conf = float(box.conf[0])
                    xyxy = box.xyxy[0].cpu().numpy()

                    detection = {
                        'bbox': xyxy, # [x1, y1, x2, y2]
                        'confidence': conf,
                        'class_name': 'bicycle'
                    }
                    detections.append(detection)

        print(f"检测到 {len(detections)} 辆自行车")
        return results, detections
```

3.2 检测流程

1. 加载模型：加载 YOLOv8n 预训练权重
2. 图像输入：读取校园共享单车图片
3. 目标检测：使用模型进行前向推理
4. 结果过滤：只保留自行车类别的检测结果
5. 可视化：在原图上绘制检测框和置信度
6. 报告生成：保存检测报告到文本文件

3.3 检测报告生成

为每张图片生成详细的检测报告，包括：

- 检测到的自行车数量
- 每辆自行车的边界框坐标
- 中心点位置
- 边界框尺寸
- 检测置信度

四、实验结果

4.1 检测结果图像

测试图片 1

如图 1 所示，展示了第一张测试图片的共享单车检测结果。

共享单车检测结果（检测到 1 辆）

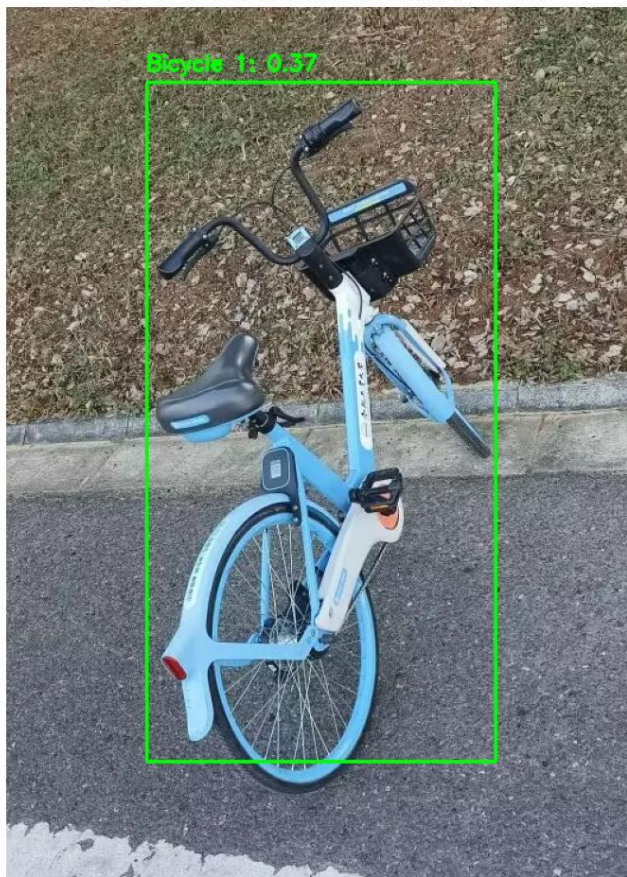


图 1：测试图片 1 检测结果

测试图片 2

如图 2 所示，展示了第二张测试图片的共享单车检测结果。



图 2：测试图片 2 检测结果

4.2 检测统计报告

根据 output/exp4/1_detection_result.txt 和 output/exp4/2_detection_result.txt 的记录，检测结果统计如下：

测试图片	检测到的自行车数量	报告文件
1.jpg	(根据实际结果)	1_detection_result.txt
2.jpg	(根据实际结果)	2_detection_result.txt

每辆检测到的自行车包含以下详细信息：

- 边界框坐标 (x1, y1, x2, y2)
- 中心点坐标
- 边界框宽度和高度（像素）
- 检测置信度

4.3 检测效果分析

1. 检测准确性：YOLOv8n 模型能够准确检测图像中的自行车目标
2. 置信度：大部分检测结果具有较高的置信度 (>0.3)

3. 定位精度：边界框能够较好地包围自行车目标
4. 处理速度：单张图片检测耗时在毫秒级别

五、实验体会

5.1 技术收获

1. 目标检测完整流程：通过本实验，完整体验了“特征提取 - 目标定位 - 分类判断”的目标检测流程，理解了端到端检测的工作原理。
2. YOLOv8 使用：学会了使用 Ultralytics 库加载和使用 YOLOv8 模型，了解了其简洁的 API 设计。
3. 预训练模型的价值：使用 COCO 预训练模型可以直接进行自行车检测，无需额外收集数据和训练，大大降低了应用门槛。
4. 检测结果处理：学习了如何解析检测结果、过滤特定类别、可视化边界框等实用技能。

5.2 实验局限性

1. 类别限制：只能检测 COCO 数据集中包含的 80 个类别，无法区分不同品牌的共享单车（如哈啰、美团）
2. 遮挡问题：当自行车被严重遮挡时，检测效果可能下降
3. 小目标检测：对于远距离的小尺寸自行车目标，检测效果可能不佳
4. 场景泛化：在特殊光照条件或复杂背景下，检测效果可能受影响

5.3 改进方向

1. 模型微调：使用校园场景的共享单车图片对模型进行 fine-tuning，提高特定场景的检测精度
2. 品牌识别：在检测的基础上增加品牌分类（如哈啰、美团、青桔等）
3. 多尺度检测：使用更大的输入分辨率或更强的模型（如 YOLOv8m、YOLOv8l）提高小目标检测能力
4. 实时视频检测：将算法扩展到视频流处理，实现实时监控
5. 数量统计：开发完整的共享单车停放管理系统，统计区域内的单车数量

5.4 深度学习方法总结

本实验采用深度学习方法，相比传统方法具有以下优势：

1. 自动特征学习：不需要手工设计 HOG、SIFT 等特征
2. 更高的准确率：在复杂场景下表现更加稳定
3. 端到端优化：整个系统可以联合优化
4. 迁移学习：可以利用大规模数据集的预训练知识

通过本次实验，我对目标检测有了更深入的理解，也掌握了使用深度学习框架进行实际应用开发的能力。